

A New Approach to Optimal Planning of Robot Motion on a Tree with Obstacles*

Vincenzo Auletta and Domenico Parente and Pino Persiano

Dipartimento di Informatica ed Applicazioni, Università di Salerno, 84081 Baronissi
(ITALY). e-mail: {auletta,parente,giuper}@dia.unisa.it

Abstract. In this paper we study the Graph Motion Planning of 1 Robot problem (GMP1R) on a tree. This problem consists in computing a minimum cost plan for moving a robot from one vertex to another in a tree whose vertices can have movable obstacles.

Papadimitriou et al. [FOCS 94] introduced the problem and gave an algorithm for the GMP1R on a tree. Their approach is based on flow arguments and yields an algorithm that solves $O(n^6)$ mincost flow problems on graphs with $O(n)$ vertices. They also give a 7 approximation algorithm that solves $O(n)$ mincost flow problems on graphs with $O(n)$ vertices.

We propose a new dynamic programming approach to GMP1R on a tree. Based on this approach we give a $O(n^4)$ algorithm for the GMP1R on a tree. Moreover, we give an $O(n^3)$ approximation algorithm that obtains a solution that is within a 7 factor from the optimum.

We also discuss extensions of our work and pose a new open problem.

1 Introduction

Consider the following optimization problem introduced in [3]. Let T be a graph with n vertices. There is a robot in a vertex s and some of the vertices of the graph contain a movable obstacle. The Graph Motion Planning of 1 Robot problem (GMP1R problem for short) consists in determining the shortest sequence of moves that takes a robot from the source vertex s to a destination vertex t , where a move consists in moving the robot or an obstacle from its current position to an adjacent vertex that does not contain any object (obstacle or robot).

The GMP1R is a particular case of the problem of planning the motion of a robot that moves in a geometric space and has to reach a destination moving obstacles out of its way if necessary. Motion planning is one of the fundamental problems in robotics. It has been extensively studied but, unfortunately, most practical versions are at least PSPACE-hard (see e.g., [5]).

The GMP1R models practical problems of robot motion in uniform settings where the robot is constrained in its movements. A typical example is that of a robot that moves on tracks and the obstacles are carts that move on the same

* Partially supported by Progetto MURST 40%, Algoritmi, Modelli di Calcolo e Struttura Informativa.

system of tracks. The robot has to make its way to the destination by moving the obstacles on the tracks. Clearly, in this and other similar settings, geometry does not play much of a role and the computational difficulty stems primarily from combinatorial considerations.

The GMP1R is related to the generalized 15-puzzle problem (see [2, 4]), where we have a graph and $n - 1$ robots to move, each with its destination, but no obstacles.

In [3] it is proved that the GMP1R is NP-complete for planar graphs. However, the simpler feasibility problem (in which we ask whether there exists a way of moving the obstacles around so to allow the robot to go from s to t) can be solved in polynomial time. In [3], the GMP1R on trees is also studied and it is proved that, in this case, the GMP1R can be solved in polynomial time. The approach presented in [3] for solving the GMP1R on a tree is based on flow arguments. It gives rise to a quite expensive algorithm that has to solve $O(n^6)$ mincost flow problems on graphs with $O(n)$ vertices. In [3], a 7-approximation algorithm for the GMP1R on a tree is given; it reduces an instance of the GMP1R on a tree to n mincost flow problems on graphs with $O(n)$ vertices.

The main results of the paper. In this paper we propose a new approach to the GMP1R on a tree. Our approach uses the properties of the canonical plans, that are particular plans introduced in [3], but in a completely different way. We use dynamic programming to reduce the GMP1R on a tree to the problem of computing a shortest path on a weighted directed acyclic graph. Our algorithms improve substantially on the previously known results.

The following theorem presents the main result of the paper.

Theorem 1. *There exists an algorithm $Plan$ that, for each instance of GMP1R on a tree that admits a solution of finite cost, computes an optimal plan in time $O(n + d^3 \cdot \min\{d^2, n\})$, where n is the number of vertices of the tree and d is the distance from the source to the destination.*

Notice that $d = O(n)$ and thus $Plan$, in the worst case, takes time $O(n^4)$. Our algorithm runs in linear time for shallow trees (for example, on complete trees or, in general, on trees of depth $O(n^{1/5})$) or on instances in which source and destination are not too far away.

The same algorithm works also for edge-weighted trees and for trees where vertices have capacities; that is, the case in which not all the edges have the same length and each vertex can host more than one obstacle. This models situations in which a better measure of the cost of a plan is given by the total length that the obstacles and the robot have to travel and where obstacles can be amassed on vertices.

Using our dynamic programming approach we can also give a $O(n^3)$ approximation algorithm for the GMP1R on a tree that computes a plan that has cost at most 7 times the optimal plan.

Organization of the paper. In Section 2 we set up our notations, give some fundamental definitions, and define the notion of *leftmost canonical plan*. We also prove that for each instance of the GMP1R that admits a finite cost solution

we can give an optimal plan that is leftmost canonical. In Section 3 we present the ideas at the base of our approach to the GMP1R. We prove that solving an instance of the GMP1R on a tree with n vertices can be reduced to solving one shortest path problem in a weighted DAG of size $O(n^6)$. An $O(n^6)$ algorithm for the GMP1R follows from the well-known fact that the shortest path problem in a DAG can be solved in time linear in the size of the DAG. In Section 4, building on the ideas presented in the previous sections, we give a more efficient reduction, that gives rise to a DAG of size $O(n^4)$. In Section 5 we study the class of the *sidestepping* plans and prove that for each instance $\mathcal{S}$ of the GMP1R on a tree there exists a sidestepping plan that solves $\mathcal{S}$ and has cost at most 7 times the cost of the optimal plan for $\mathcal{S}$. Then we show how to compute in time $O(n^3)$ the optimal sidestepping plan. Finally, in Section 6 we discuss some extensions of our work and some open problems.

2 Notation and definitions

An instance $\mathcal{S}$ of the GMP1R on a tree (in the following referred to as TMP1R) consists of a tree T , two distinct vertices s and t called the source and destination vertex, and a subset S of the vertices of T , distinct from s , that contain an obstacle. The TMP1R consists in computing a plan of minimum number of moves for moving a robot from the source to the destination vertex.

Example 1. Figure 1 describes an instance $\mathcal{S}$ of the TMP1R. A possible plan A for $\mathcal{S}$ first moves the robot from vertex s to vertex u_3 , and the obstacles in v_1, v_2, v_3, v_4, v_5 and v_6 to u_1, u_2, u_6, u_8, u_9 and v_9 , respectively. Then, it moves the robot to u_7 and the obstacles in v_7, v_8 and v_9 to v_3, v_4 and v_5 . Finally, it moves the robot from u_7 to the destination vertex. It can be easily seen that A is an optimal plan for $\mathcal{S}$.

Throughout this paper n denotes the number of vertices of T and d the length of the path $P = (v_0, \dots, v_d)$ from the source vertex $s = v_0$ to the destination vertex $t = v_d$. We call P the *critical path* of $\mathcal{S}$ and call a vertex of P distinct from s with at least one neighbour not on P a *branch vertex*. For each branch vertex v_j we define the *sidestep vertex* of v_j as an arbitrary but fixed neighbour of v_j not on P .

For a vertex v we denote by $F(v)$ the forest obtained from T by removing the vertex v . For a forest F we denote by $T(F, v)$ the tree of the forest F containing the vertex v . For any pair of vertices u, v , if u does not belong to $T(F(t), v)$ we say that u is *behind* v and that v *follows* u .

We organize the vertices of the tree T in the following way (see Figure 2). For each vertex v_j of P , with $j > 0$, we let B_j be the set of vertices of $F(j)$ that are neither in $T(F(j), s)$ nor in $T(F(j), t)$. That is, B_j consists of the vertices of the trees rooted in the neighbours of v_j which are not on the critical path. We notice that if v_j is not a branch vertex then B_j is empty. Moreover, for each vertex v_j in P and each branch vertex v_r , with $0 < r < j$, we denote by $\text{Ch}(r, j)$ the set of vertices of P between v_r and v_j (v_j excluded). $\text{Ch}(0, j)$, instead, denotes the

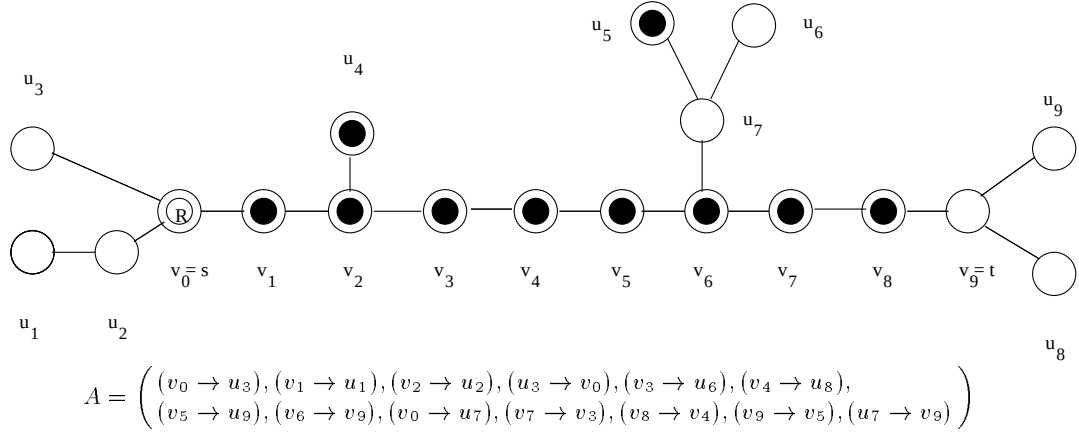


Fig. 1. An instance $\mathcal{S}$ of GMP1R on a tree and a sequence of moves A that solves $\mathcal{S}$.

set of the vertices of P between s and v_j (v_j excluded) and the vertices behind s . We say that a vertex is a hole if it does not have an obstacle. If a path from

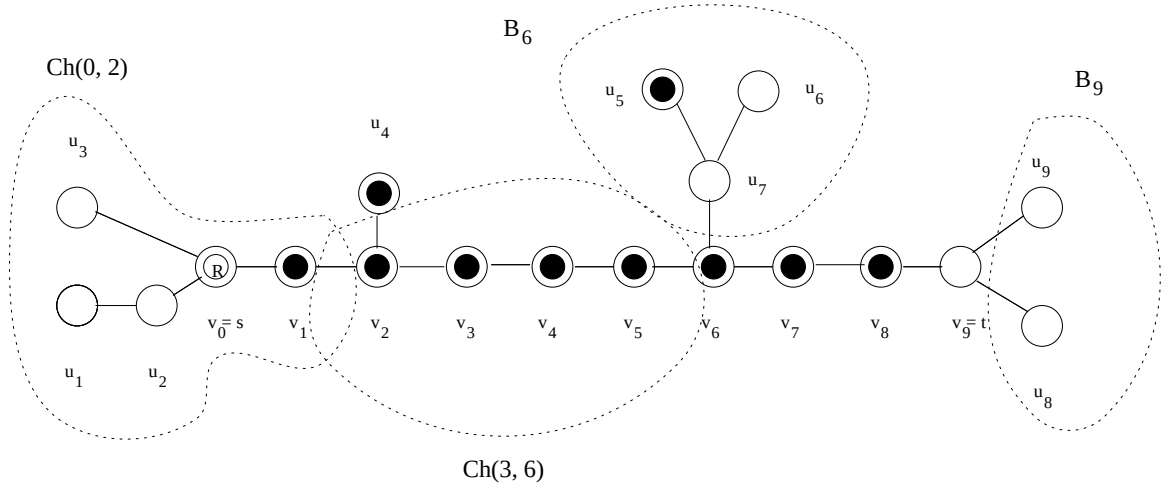


Fig. 2. The organization of the tree.

u to v is filled with obstacles except for v , each of these obstacles can be moved one step towards v . We can view the net effect as moving an obstacle from u to v . This holds also if there are holes between u and v . Thus we allow an obstacle to traverse a path not containing the robot in one move and charge the length of the path as cost of the move.

2.1 Leftmost canonical plans

We introduce the notion of leftmost canonical plan which is a particular type of the canonical plans introduced in [3].

We call a sequence of moves that takes the robot from the source vertex to the destination vertex a *quasi-bitonic* plan if it can be divided in two parts that satisfy the following conditions. In the first part, that we call *backup part*, the robot visits only vertices not belonging to $T(F(s), t)$; in the second part, that we call *forward part*, the robot goes from s to t . Moreover, in the forward part the robot on arriving at an internal vertex v of P , either moves directly to the next vertex in P or, if v is a branch vertex, it moves to the sidestep of v , comes back v and then proceeds to the next vertex in P .

Let P_{for} and P_{bac} be the set of vertices visited by the robot during the forward and backup parts of the plan, respectively. An obstacle in P_{for} is moved *outward* if it is moved to a vertex in $T(F(s), t)$ but not in P_{for} ; *forward* if it is moved to a vertex in P_{for} following its position; *backward* if it is moved to a vertex of P_{for} that is behind its position or to a vertex which is behind s . If an obstacle is moved from vertex v to vertex u by plan A , then we assume that the moves that take the obstacle from v to u appear consecutively in A . We say that such a sequence *originates* in v and *ends* in u and is a backward, forward or outward sequence depending on whether the obstacle movement is backward, forward, or outward, respectively. If a move is part of a backward, forward, or outward sequence, then we say that the move itself is a backward, forward, or outward move, respectively.

Example 2. Consider the plan A given in Figure 1. We have that P_{bac} contains the vertices u_1, u_2, u_3 while P_{for} contains the vertices $v_0, v_1, \dots, v_9$ and u_7 . The backup part of A consists of the moves of the robot from v_0 to u and from u_3 to v_0 , and of the moves of the obstacles in v_1 and v_2 to u_1 and u_2 . The moves of the obstacles in v_3, v_4 and v_5 to u_6, u_8 and u_9 are outward moves; the move of the obstacle in v_6 to v_9 is a forward move; the moves of the obstacles in v_7, v_8 and v_9 to v_3, v_4 and v_5 are backward moves.

In [3] a *canonical* plan is defined as a quasi-bitonic plan that satisfies the following additional conditions on the moves of the obstacles:

- no obstacle placed outside P_{for} and P_{bac} is ever moved;
- each obstacle initially at vertices of P_{for} is moved once (outward or backward) or twice (first forward and then backward);
- if an obstacle is moved backward, then it traverses at least a branch vertex;
- each obstacle in P_{bac} is moved out of P_{bac} before the robot leaves s ;
- all forward and outward moves are performed before the robot leaves s .

In [3] it is proved that for each instance of the TMP1R that admits a solution of finite cost, there exists an optimal plan that is canonical.

We introduce a new class of plans, that we call *leftmost canonical plans*.

Definition 2. A canonical plan A is a *leftmost canonical plan* if the following two conditions hold:

1. A performs all forward moves before of any outward move;
2. for each pair of vertices u, w of P_{for} , with u behind w , if A moves an obstacle from w to a vertex behind u then it moves the obstacle in u either to a vertex behind u or forward to a vertex of P_{for} following w .

The next lemma proves that for each instance $\mathcal{S}$ of the TMP1R that admits a finite cost solution there exists an optimal plan that is leftmost canonical. Therefore, in this paper we will restrict our attention to leftmost canonical plans.

Lemma 3. *For each instance $\mathcal{S}$ of the TMP1R that admits a finite cost solution, there exists an optimal plan for $\mathcal{S}$ that is leftmost canonical.*

3 The dynamic programming approach

In this section we give a simple $O(n^6)$ algorithm for the TMP1R. Although inefficient this algorithm contains all the ideas at the base of our dynamic programming approach to TMP1R. We postpone efficiency considerations to the next section where we show how to improve the algorithm for the TMP1R to obtain a running time of $O(n^4)$.

For sake of clarity in the sequel we assume that there is no vertex of degree greater than 2 behind the source vertex s (thus, the backup part of the plan is empty) and no obstacle is originally placed in the sidestep vertices of P . Moreover, we assume that the plan cannot contain moves that take obstacles in the sidestep vertices. Extending our approach to the general case is only a matter of technicalities and will be presented in the full version of the paper.

At the base of our approach is the idea of partitioning the set of moves of a plan A for solving an instance $\mathcal{S}$ of the TMP1R, with respect to a vertex v_j of the critical path, into two sets $L(A, j)$ and $R(A, j)$.

Definition 4. Let $\mathcal{S}$ be an instance of the TMP1R and let $P = (v_0, \dots, v_d)$ be its critical path. For each $0 \leq j \leq d$, we define the partition $(L(A, j), R(A, j))$ of a plan A with respect to v_j as follows. $L(A, j)$ consists of the following moves of A :

- all the robot moves that originate in a vertex of $T(F(j), s)$;
- all forward and outward moves that end in v_j or in a vertex behind v_j ;
- all moves belonging to backward sequences that end in vertices in $T(F(r), s)$, where v_r is either the rightmost branch vertex at which the robot took a sidestep before arriving to v_j , or the source if the robot took no sidestep before arriving to v_j ;
- all moves belonging to forward sequences that end in vertices between v_j and v_l , where v_l is the rightmost vertex which is the origin of either an outward sequence ending behind v_j or a backward sequence ending in $T(F(r), s)$.

$R(A, j)$ consists of all the remaining moves of A .

Thus, $R(A, j)$ consists of all robot and outward moves that end in vertices that follow v_j ; the robot move from v_j to its sidestep (if it has any) and back; all forward moves that end in vertices following v_j and belonging to forward sequences

ending in vertices following v_i ; all moves belonging to backward sequences ending in $\text{Ch}(r, j)$ or in vertices of P_{for} following v_{j-1} .

We call $L(A, j)$ and $R(A, j)$ the left and right part of the plan A with respect to v_j . We say that a set of moves R is a legal completion of a left part L with respect to $\mathcal{S}$ if $R \cup L$ is a plan for $\mathcal{S}$. The cost of the legal completion R is simply the number of moves of R . The minimum legal completion R^* of L is the legal completion of L with minimum cost.

Example 3. In figure 3 it is described an optimal leftmost canonical plan A . In order to compute the left part of A with respect to v_5 we notice that v_4 is the rightmost branch vertex where the robot took a sidestep before arriving to v_5 and v_9 is the rightmost vertex that is origin of a sequence ending behind v_5 . The left part of A with respect to the vertex v_5 consists of the robot moves from v_0 to v_5 (including the sidestep in u_1), the outward sequence from v_1 to u_3 , the forward sequence from v_4 to v_9 , the backward sequence from v_9 to v_3 , the outward moves from v_2 and v_3 to v_5 (notice that these moves belong to outward sequences ending in u_4 and u_5). Obviously, $R(A, 4)$ consists of all the remaining moves of A .

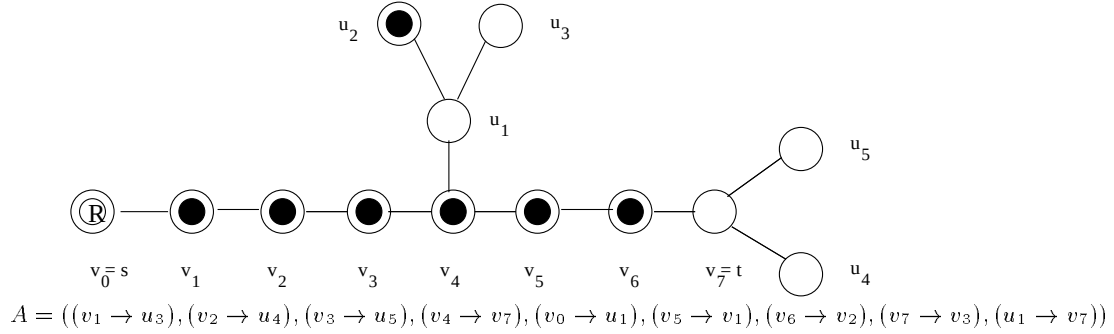


Fig. 3. An instance $\mathcal{S}$ of TMP1R and a leftmost canonical plan A that solves $\mathcal{S}$.

Let $\mathcal{S}$ be an instance of TMP1R. For each vertex v_j , we partition the set consisting of the left parts $L(A, j)$, where A is a leftmost canonical plan for $\mathcal{S}$, into classes according to three parameters. We say that the left part $L(A, j)$ relative to v_j belongs to the class $\mathcal{C}(j, k, l, r)$ if it contains k obstacle moves that end in v_j ; v_r is either the rightmost branch vertex behind v_j at which the robot took a sidestep before arriving in v_j , or the source if the robot took no sidestep before arriving to v_j ; v_l is the rightmost origin of either an outward sequence ending behind v_j , or a backward sequence ending in $T(F(r), s)$.

A left part of class $\mathcal{C}(j, k, l, r)$ can be seen as a partial plan that takes the robot from s to v_j but has not decided about the final destination of k obstacles

(which we call the *suspended* obstacles) that were present on the path from s to v_j . However, it is decided that each of these obstacles will be moved either outward to some B_i with $i > j$ or first forward to a vertex v_z with $z > l$ and then backward to a vertex of $(v_r, \dots, v_j)$. For each left part L relative to v_j our algorithm computes the minimum cost legal completion of L in terms of the minimal cost legal completions of left parts relative to v_{j+1} .

Definition 5. Let $L \in \mathcal{C}(j, k, l, r)$ and $L' \in \mathcal{C}(j+1, k', l', r')$. Then, we say that L' is reachable from L , and that the set of moves M is an extension of L into L' , if and only if $L' = L \cup M$ and, for each move $m \in M$ and for each leftmost canonical plan A , $L \cup \{m\} \not\subseteq L(A, j)$.

An extension M of the left part $L \in \mathcal{C}(j, k, l, r)$ into $L' \in \mathcal{C}(j+1, k', l', r')$ has a particular structure that we now characterize. M can contain sequences of obstacle moves that end behind v_{j+1} ; but, if this is so, these sequences can be either outward sequences ending in B_{j+1} or in case v_j is a branch vertex, backward sequences ending in $\text{Ch}(r, j)$. Moreover, any legal completion of L' does not contain outward sequences to B_{j+1} . In other words, if the plan A moves obstacles to vertices of B_{j+1} , it does so in the extension of $L(A, j)$ into $L(A, j+1)$. Similarly, if the robot goes to the sidestep of v_j and moves obstacles backward to $\text{Ch}(r, j)$, then any legal completion of L' does not contain any other backward sequence ending in $\text{Ch}(r, j)$. The next lemma summarizes the structure of an extension.

Lemma 6. Let $L \in \mathcal{C}(j, k, l, r)$ and $L' \in \mathcal{C}(j+1, k', l', r')$ be the left part reached from L by the extension M . M contains only moves of the following types:

- 1) robot move from v_j to v_{j+1} ;
- 2) obstacle moves from v_j to v_{j+1} ;
- 3) outward sequences to B_{j+1} ;
- 4) if v_j is a branch vertex and M contains robot moves from v_j to its sidestep and back then M contains forward sequences from v_j to vertices v_m , with $l \leq m < l'$, and backward sequences originating from the vertices $v_{l+1}, \dots, v_{l'}$ and each ending to a vertex of $\text{Ch}(r, j)$.

We say that an extension M of L into L' is a (h_1, h_2) -extension if M contains h_1 outward sequences and h_2 backward sequences. To compute the best (h_1, h_2) -extension of a left part $L \in \mathcal{C}(j, k, l, r)$ we need to decide on which obstacles are moved outward and backward and where they are moved.

Lemma 7. Each instance of the TMP1R that admits a finite cost solution has an optimal leftmost canonical plan A such that, for each $0 \leq j \leq d$, the extension M_j of $L(A, j)$ into $L(A, j+1)$ has the following three properties.

- Suppose that a suspended obstacle is moved backward by M_j . Then no obstacle following v_l is moved outward to B_{j+1} by M_j .
- Suppose that a suspended obstacle is moved forward to a vertex v_z , with $z > l$, and then backward by M_j . Then all obstacles between v_{l+1} and v_z are also moved backward by M_j .

- Suppose that no suspended obstacle is moved forward by M_j to a hole v_z with $l < z$. Then no suspended obstacle is moved forward to a vertex v_s with $s > z$ by M_j .

Proof. We prove only property 1, the proofs of the other two properties being similar. Suppose that a suspended obstacle is moved first forward to v_z and then backward to v_x and that an obstacle is moved from v_y to vertex v^* in B_{j+1} . Then the extension that moves the suspended obstacle to v^* and the obstacle in v_y to v_x has a no greater cost.

3.1 Reducing TMP1R to shortest path on a weighted DAG

By Lemma 7 we have that the obstacles to be moved in the minimal cost (h_1, h_2) -extension can be selected in the following way. If $h_1 < k$ then h_1 suspended obstacles are moved to the h_1 closest holes of B_{j+1} ; the h_2 obstacles to be moved backward are selected as follows. The vertices of P are scanned starting from v_{l+1} . If the current scanned vertex v_m has an obstacle, then this obstacle is taken and moved back to the current closest hole of $\text{Ch}(r, j)$ (in the actual plan these moves are performed while the robot is at the sidestep vertex of v_j). If the current vertex v_m has a hole and there are still some suspended obstacles, then one suspended obstacle is moved forward to v_m and then backward to the current closest hole of $\text{Ch}(r, j)$. Instead if v_m has a hole and there are no suspended obstacles left then the algorithm skips v_m .

On the other hand, if $h_1 > k$ then the k suspended obstacles are moved to the k closest holes of B_{j+1} . The remaining $h_1 - k$ obstacles to be moved outward and the h_2 obstacles to be moved backward are the $h_2 + h_1 - k$ rightmost obstacles in vertices of P ahead of v_l .

From the discussion above, it is evident that the minimal cost (h_1, h_2) -extension of a left part $L \in \mathcal{C}(j, k, l, r)$ can be computed from the knowledge of the parameters j, k, l , and r and the location of obstacles along the path from v_l to t and in $\text{Ch}(r, j)$ that is the same for all left parts of $\mathcal{C}(j, k, l, r)$. We can conclude that all left parts of $\mathcal{C}(j, k, l, r)$ have the same minimal-cost extensions and thus they all have the same minimum cost legal completion. In the sequel we say that a class C_1 reaches the class C_2 by the extension M if and only if there is a left part $L_1 \in C_1$ and a left part $L_2 \in C_2$ such that L_1 reaches L_2 by M .

The graph $G(\mathcal{S})$. For each instance $\mathcal{S}$ of the TMP!R, we construct a weighted directed graph $G(\mathcal{S})$ that has a vertex $w(j, k, l, r)$ for each non empty class $\mathcal{C}(j, k, l, r)$ and two distinguished vertices w_s and w_t that we call source and sink. The source w_s is connected to $w(0, 0, 0, 0)$ by an edge with empty label and length 0 and all the vertices $w(d, 0, d, r)$ are connected to w_t by edges with empty label and length 0. Moreover, there is a directed edge from $w(j, k, l, r)$ to $w(j', k', l', r')$ if and only if $\mathcal{C}(j', k', l', r')$ is reachable from $\mathcal{C}(j, k, l, r)$. This edge is labelled with a compact coding of the minimum cost extension from $\mathcal{C}(j, k, l, r)$ to $\mathcal{C}(j', k', l', r')$ (i.e., if the minimum cost extension is a (h_1, h_2) -extension then the edge is labelled with the pair (h_1, h_2)) and its length is set equal to the

cost of this extension. As by the definition of extension there exists no edge in $G(\mathcal{S})$ connecting vertices $w(j, k, l, r)$ and $w(j', k', l', r')$ with $j' \neq j + 1$, $G(\mathcal{S})$ is a directed acyclic graph (a DAG).

Each path from the source to the sink of $G(\mathcal{S})$ naturally defines a leftmost canonical plan for $\mathcal{S}$ obtained by considering the moves of the labels of the edges of the path. The cost of the plan is given by the sum of the lengths of the edges of the path.

Similarly, each leftmost canonical plan A is associated with a path of $G(\mathcal{S})$. For each $0 \leq j \leq d$, denote by C_j the class such that $L(A, j) \in C_j$. Then, there exists a path in $G(\mathcal{S})$ that visits the vertices corresponding to the classes C_j . The cost of this path is equal to the cost of the best leftmost canonical plan among the plans A^* such that $L(A^*, j) \in C_j$, for each j . Thus, our problem is reduced to that of computing the shortest path in $G(\mathcal{S})$ from the source to the sink. The shortest path problem in a DAG can be solved in time proportional to the number of its edges and vertices. Next we prove how to compute the minimum cost (h_1, h_2) -extension of a class and that $G(\mathcal{S})$ has $O(d^4)$ vertices and $O(d^6)$ edges.

Computing out and back. To compute the minimum cost (h_1, h_2) -extension we use the functions $out(j, k, l, h)$ and $back(j, k, l, r, h)$ defined as follows. $out(j, k, l, h_1)$ is defined as the minimum cost of moving h_1 obstacles outward to B_{j+1} in an extension of a left part of $\mathcal{C}(j, k, l, r)$ (actually, this is independent from r). Similarly, $back(j, k, l, r, h_2)$ is defined to be the minimum cost of moving h_2 obstacles backward to $Ch(r, j)$ in an extension of $\mathcal{C}(j, k, l, r)$. Both functions return, besides the actual minimal cost c of performing the moves, the number k^* of remaining suspended obstacles and the index l^* of the rightmost vertex of P from which an obstacle has been moved behind v_{j+1} .

The values of $back$ and out are computed using the table of the positions of the obstacles and holes in the starting configuration. This table can be precomputed in time $O(n)$. The computation of the cost returned by out is carried out in the following way. Obviously, $out(j, k, l, 0) = 0$. The value of $out(j, k, l, h_1 + 1)$ is computed in constant time from the value of $out(j, k, l, h_1)$ and the knowledge of the location of the $(h + 1)$ -st closest hole of B_{j+1} and, if $h + 1 > k$, the location of the $(k - h_1 - 1)$ -st leftmost obstacle ahead of v_l . If B_{j+1} has less than $h_1 + 1$ holes or there are less than $h + 1$ obstacles between suspended obstacles and obstacles in vertices ahead of v_l then, obviously, $out(j, k, l, h_1 + 1)$ and all subsequent values are set equal to ∞ . The values k^* and l^* are computed similarly. We can thus conclude that, for each j, k and l , the values of $out(j, k, l, h_1)$ for all needed h_1 's can be computed in time $O(\min\{d, |B_{j+1}|\})$. Similarly, for each j, k, l and r all the needed values of $back(j, k, l, r, h_2)$ are computed in an incremental way in time $O(\min\{d, |Ch(r, j)|\})$.

Computing the cost of the optimal plan. Assuming that all needed values of out and $back$ are computed in a preprocessing phase, it is easy to give a constant time algorithm $EXTEND(j, k, l, r, h_1, h_2)$ (see Figure 4) that uses the functions out and $back$ to compute the minimum cost (h_1, h_2) -extension M of a class $\mathcal{C}(j, k, l, r)$

and the class reached from $\mathcal{C}(j, k, l, r)$ by M . The algorithm EXTEND is used by algorithm LIST (see Figure 4) to list all the edges directed out from a vertex $w(j, k, l, r)$ along with their labels and lengths. We use the algorithm LIST in the construction of the graph $G(\mathcal{S})$. For each vertex w of the graph we consider the class corresponding to w and use LIST to list the classes reachable from w and compute the labels and the lengths of the corresponding edges.

EXTEND(j, k, l, r, h_1, h_2)

Input: A 4-tuple (j, k, l, r) describing a class of left parts and two integers h_1 and h_2 .

begin

($c_{h_1}, k_{h_1}, l_{h_1}$) $\leftarrow$ out(j, k, l, h_1);
 ($c_{h_2}, k_{h_2}, l_{h_2}$) $\leftarrow$ back($j, k_{h_1}, l_{h_1}, r, h_2$);
 $c \leftarrow c_{h_1} + c_{h_2} + (k - k_{h_2} + 1)$;
if ($h_2 > 0$) **then** $r_{h_2} \leftarrow j$ **else** $r_{h_2} \leftarrow r$;
return ($c, \mathcal{C}(j + 1, k_{h_2}, l_{h_2}, r_{h_2})$);

end

LIST(j, k, l, r)

Input: A 4-tuple (j, k, l, r) describing a vertex of $G(\mathcal{S})$.

begin

Set $\mathcal{L} = \emptyset$ and compute all values of *out* and *back* needed.
 Set B and C equal to the number of holes of B_{j+1}
 (distinct from the sidestep of v_{j+1}) and $\text{Ch}(r, j)$, respectively.
if v_j is a branch vertex **then** $p \leftarrow C$ **else** $p \leftarrow 0$
for $h_1 = 0$ **to** $\min\{d, B\}$
 for $h_2 = 0$ **to** $\min\{d, p\}$
 ($c, \mathcal{C}(j + 1, k^*, l^*, r^*)$) $\leftarrow$ EXTEND(j, k, l, r, h_1, h_2)
if $\mathcal{C}(j + 1, k^*, l^*, r^*) \notin \mathcal{L}$ **then** $\mathcal{L} \leftarrow \mathcal{L} \cup \{(c, (h_1, h_2), \mathcal{C}(j + 1, k^*, l^*, r^*))\}$
else update the extension into $\mathcal{C}(j + 1, k^*, l^*, r^*)$ to the minimum cost extension between
 (h_1, h_2)-extension and the extension previously computed.

return $\mathcal{L}$.

end

Fig. 4. The algorithms EXTEND and LIST.

We next prove that LIST takes time $O(d^2)$.

Lemma 8. *The algorithm LIST on input (j, k, l, r) lists all the classes reachable from $\mathcal{C}(j, k, l, r)$ and computes the corresponding minimal cost extensions, along with its cost, in time $O(d^2)$.*

Proof. By Lemma 6 it follows that an extension of $\mathcal{C}(j, k, l, r)$ moves at most $\min\{d, |\text{Ch}(r, j)| + |B_{j+1}|\}$ obstacles behind v_{j+1} . These obstacles can be moved outward or backward and for each number of obstacles moved outward and backward we have a different extension. Thus, the extensions of $\mathcal{C}(j, k, l, r)$ and

the corresponding values of *out* and *back* that need to be computed are $O(d^2)$ and can be computed in time $O(d^2)$.

Theorem 9. *An optimal leftmost canonical plan for an instance $\mathcal{S}$ of TMP1R can be obtained by solving a shortest path problem in a weighted DAG with $O(d^6)$ edges and $O(d^4)$ vertices.*

Proof. By the previous discussion we have that there is a plan for $\mathcal{S}$ if and only if there exists a path between the source and the sink of the weighted DAG $G(\mathcal{S})$. Moreover, the optimal plan for $\mathcal{S}$ is obtained by the shortest source-sink path of $G(\mathcal{S})$ by taking the moves of the labels associated to the edges of the path.

By Lemma 8 each of the $O(d^4)$ vertices of $G(\mathcal{S})$ has $O(d^2)$ outgoing edges.

Corollary 10. *There exists an algorithm that, for each instance of the TMP1R that admits a solution of finite cost, computes the cost of an optimal plan in time $O(n + d^6)$.*

4 A more efficient algorithm for TMP1R

The algorithm given in the previous section solves an instance $\mathcal{S}$ of TMP1R by constructing a weighted directed acyclic graph $G(\mathcal{S})$ with $O(d^4)$ vertices and $O(d^6)$ edges and solving a shortest path problem on $G(\mathcal{S})$. In this section we give the construction of a directed acyclic graph $H(\mathcal{S})$ that has $O(d^4)$ vertices, $O(d^3 \cdot \min\{n, d^2\})$ edges and still encodes the TMP1R instance $\mathcal{S}$ so that a path in this graph corresponds to a leftmost canonical plan for $\mathcal{S}$ and the shortest of such paths corresponds to an optimal plan for $\mathcal{S}$. We shall prove that $H(\mathcal{S})$ can be constructed in time $O(n + d^3 \cdot \min\{n, d^2\})$.

We will obtain $H(\mathcal{S})$ in two steps. We first show how to construct a graph $J(\mathcal{S})$ that has $O(d^3(n + d^2))$ edges and still encodes the TMP1R instance $\mathcal{S}$. Then, we show how $H(\mathcal{S})$ can be obtained from $J(\mathcal{S})$.

4.1 The graph $J(\mathcal{S})$

The following observation is at the base of the construction of $J(\mathcal{S})$. Each extension M can be thought of as consisting of two parts: the out extension, which includes all the outward sequences of the extension, and the back extension, which includes all the other moves (robot moves, forward and backward sequences, etc.). This splitting of the extensions leads to a partitioning of a leftmost canonical plan that is a refinement of the partition presented in the previous section. A plan A is partitioned with respect to two parameters: a vertex v_j of the critical path P and a bit b . The left part $L(A, j, 0)$ of A with respect to v_j and 0 is defined as equal to $L(A, j)$. Whereas, we define the left part $L(A, j, 1)$ of A with respect to v_j and 1 as consisting of all the moves of $L(A, j)$ and all the outward sequences of A that end in vertices of B_{j+1} . As before, the set of all the possible left parts of leftmost canonical plans for $\mathcal{S}$ relative to a vertex v_j and b can be partitioned in classes according to three parameters k, l and r .

Definition 11. For any leftmost canonical A the left part $L(A, j, b)$ belongs to the class $\mathcal{C}(j, k, l, r, b)$ if and only if the following conditions hold: it contains k obstacle moves that end in v_j ; v_r is either the rightmost branch vertex behind v_j at which the robot took a sidestep before arriving in v_j or the source vertex if the robot took no sidesteps before arriving to v_j ; v_l is the rightmost vertex origin of either an outward sequence ending behind v_{j+1} or a backward sequence ending in $T(F(r), s)$.

We can now define the notion of out and back extensions. If $L \in \mathcal{C}(j, k, l, r, 0)$ and $L' \in \mathcal{C}(j, k', l', r, 1)$, then we say that L' is *out reachable* from L , and that the set of moves M is an *out extension* of L into L' , if and only if $L' = L \cup M$ and, for each move $m \in M$ and for each leftmost canonical plan A , $L \cup \{m\} \not\subseteq L(A, j, 0)$. Similarly, if $L \in \mathcal{C}(j, k, l, r, 1)$ and $L' \in \mathcal{C}(j+1, k', l', r', 0)$, then we say that L' is *back reachable* from L , and that the set of moves M is a *back extension* of L into L' , if and only if $L' = L \cup M$ and, for each move $m \in M$ and for each leftmost canonical plan A , $L \cup \{m\} \not\subseteq L(A, j, 1)$.

As done before for $G(\mathcal{S})$, we let the graph $J(\mathcal{S})$ have a vertex $u(j, k, l, r, b)$ for each non-empty class $\mathcal{C}(j, k, l, r, b)$ and one source and one sink vertex. Moreover, there is an edge from the vertex corresponding to class $\mathcal{C}_1$ to the vertex corresponding to class $\mathcal{C}_2$ if and only if $\mathcal{C}_2$ is out or back reachable from $\mathcal{C}_1$ and a zero-weight edge from the source vertex to $u(0, 0, 0, 0, 0)$ and from each vertex $u(d, 0, d, r, 0)$ to the sink vertex.

Lemma 12. *The weighted DAG $J(\mathcal{S})$ associated with the instance $\mathcal{S}$ of TMP1R has $O(d^3(n + d^2))$ edges.*

Proof. We notice that the class $\mathcal{C}(j, k, l, r, 0)$ has $O(\min\{d, |B_{j+1}|\})$ distinct out extensions and the class $\mathcal{C}(j, k, l, r, 1)$ has $O(\min\{d, |\text{Ch}(r, j)|\})$ distinct back extensions. Moreover, each of these extensions reaches a different class. Summing over all the vertices of $J(\mathcal{S})$ we obtain that the number of edges is

$$\sum_{j=0}^{d-1} O(d^3 \cdot \min\{d, |B_{j+1}|\}) + \sum_{j=0}^{d-1} O(d^3 \cdot \min\{d, |\text{Ch}(z, j)|\}) + O(d) = O(d^3(d^2 + n)).$$

4.2 The graph $H(\mathcal{S})$

The number of edges of the graph that encodes the TMP1R instance $\mathcal{S}$ can be further reduced by observing that some back extensions can be ignored in computing the optimal plan for $\mathcal{S}$. Consider two left parts L_r and L_z belonging to the classes $\mathcal{C}(j, k, l, r, 1)$ and $\mathcal{C}(j, k, l, z, 1)$, where v_z is the leftmost branch vertex along the path $(v_{r+1}, \dots, v_{j-1})$. Moreover, let M_r and M_z be back extensions of L_r and L_z , respectively, that move the same number $h > 0$ of obstacles backward. It is easy to see that, if $1 \leq h \leq |\text{Ch}(z, j)|$, M_r and M_z are equal and both reach left parts of the same class $\mathcal{C}(j+1, k', l', j, 0)$. Thus, each legal completion of L_z that moves some obstacles backward while the robot is at the sidestep of v_j is also a legal completion of L_r . The case $h = 0$, in which the robot does not take

a sidestep in v_j , is different: in this case, L_r is extended by M_r into a left part of $\mathcal{C}(j+1, k', l', r, 0)$ and L_z is extended by M_z into a left part of $\mathcal{C}(j+1, k', l', z, 0)$. However, next lemma proves that the minimal legal completion of $\mathcal{C}(j, k, l, r, 1)$ has cost no greater than the minimal legal completion of $\mathcal{C}(j, k, l, z, 1)$.

Lemma 13. *Let $L_r \in \mathcal{C}(j, k, l, r, b)$ and $L_z \in \mathcal{C}(j, k, l, z, b)$, where v_z is the leftmost branch vertex belonging to $(v_{r+1}, \dots, v_{j-1})$. It holds that the minimal cost legal completion of L_r has cost not greater than the minimal cost legal completion of L_z .*

By Lemma 13, the minimal legal completion of $\mathcal{C}(j, k, l, r, 1)$ is obtained by taking the minimum among the minimal legal completion of $\mathcal{C}(j, k, l, z, 1)$ and the minimal legal completion obtained by back extensions of $\mathcal{C}(j, k, l, r, 1)$ that move 0 or more than $|\text{Ch}(z, j)|$ obstacles backward (we remark that these are back extensions of $\mathcal{C}(j, k, l, r, 1)$ but not of $\mathcal{C}(j, k, l, z, 1)$). Thus, the graph $H(\mathcal{S})$ is obtained from $J(\mathcal{S})$ by removing for each vertex $u(j, k, l, r, 1)$ the edges labelled with back extensions that move h obstacles backward, for $1 \leq h \leq |\text{Ch}(z, j)|$ where v_z is the leftmost branch vertex in $(v_{r+1}, \dots, v_{j-1})$, and adding a new edge from $u(j, k, l, r, 1)$ to $u(j, k, l, z, 1)$ with empty label and length 0 (see Figure 5). We observe that there exists a path in $H(\mathcal{S})$ between the vertices $u(j, k, l, r, 0)$

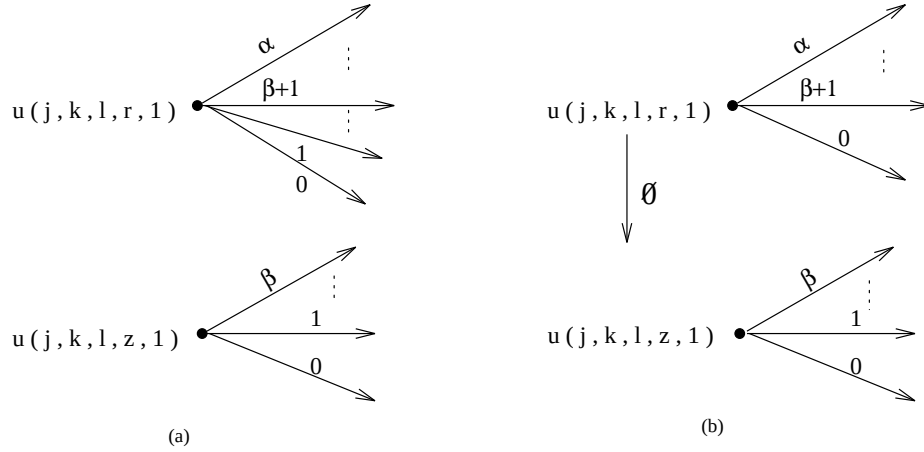


Fig. 5. Two nodes of $H(\mathcal{S})$. For each edge we reported a compact coding of the corresponding extensions, where $\alpha = |\text{Ch}(r, j)|$ and $\beta = |\text{Ch}(z, j)|$.

and $u(j+1, k', l', r', 0)$ if and only if there exists an edge in $G(\mathcal{S})$ between the vertices $w(j, k, l, r)$ and $w(j+1, k', l', r')$. The length of the shortest path from $u(j, k, l, r, 0)$ to $u(j+1, k', l', r', 0)$ and the union of the labels of its edges are equal to the length and the label of the edge from $w(j, k, l, r)$ to $w(j+1, k', l', r')$. This implies that there is a source-sink path in $H(\mathcal{S})$ if and only if there is a source-sink path in $G(\mathcal{S})$ and the shortest paths in the two graphs have the

same length and labels. Thus, the problem of computing an optimal plan for an instance $\mathcal{S}$ of TMP1R is reduced to the problem of computing the shortest path between the source and the sink of the weighted directed acyclic graph $H(\mathcal{S})$. We prove that $H(\mathcal{S})$ is acyclic, that it has $O(d^3 \cdot \min\{n, d^2\})$ edges and it can be constructed in time $O(n + d^3 \cdot \min\{n, d^2\})$. Thus, we can conclude that there is an $O(n + d^3 \cdot \min\{n, d^2\})$ algorithm that, for any instance $\mathcal{S}$ of the TMP1R that admits a finite cost solution, computes a minimal cost leftmost canonical plan for $\mathcal{S}$.

Lemma 14. *For each instance $\mathcal{S}$ of the TMP1R, the graph $H(\mathcal{S})$ is acyclic and it has $O(d^3 \cdot \min\{n, d^2\})$ edges.*

Proof. We observe that, since the source u_s has indegree 0 and the sink u_t has outdegree 0, there is no cycle in $H(\mathcal{S})$ containing u_s or u_t . Thus, to prove that $H(\mathcal{S})$ is a DAG it is sufficient to give a partial ordering $\leq$ on the classes $\mathcal{C}(j, k, l, r, b)$ such that for each edge $(u(j, k, l, r, b), u(j', k', l', r', b'))$ in $H(\mathcal{S})$ it holds that $\mathcal{C}(j, k, l, r, b) \leq \mathcal{C}(j', k', l', r', b')$.

Let $\mathcal{C}(j, k, l, r, b) \leq_{\mathcal{S}} \mathcal{C}(j', k', l', r', b')$ if and only if $j = j'$, $k = k'$, $l = l'$, $r = r'$ and $b = b'$, or

$$\begin{array}{ll} j < j' & \text{or} \\ j = j' \text{ and } b < b' & \text{or} \\ j = j' \text{ and } b = b' \text{ and } r > r'. \end{array}$$

It can be easily seen that $\leq_{\mathcal{S}}$ satisfies our requirement.

Let us count the number of the edges going out of vertices of $H(\mathcal{S})$. Vertex $u(j, k, l, r, 0)$, for $j < d$, has $O(\min\{d, |B_{j+1}|\})$ outgoing edges, corresponding to the out extensions of $\mathcal{C}(j, k, l, r, 0)$. Instead, vertex $u(d, k, l, r, 0)$ has at most one outgoing edge. Let z be the leftmost branch vertex along the path $(v_{r+1}, \dots, v_{j-1})$. Vertex $u(j, k, l, r, 1)$ has $O(\min\{d - |\text{Ch}(z, j)|, |\text{Ch}(r, z)|\})$ outgoing edges, one for each back extension of $\mathcal{C}(j, k, l, r, 1)$ which moves 0 or more than $|\text{Ch}(z, j)|$ obstacles backward, and one edge of weight 0 directed to $u(j, k, l, z, 1)$. The source u_s has one outgoing edge.

Summing over all the vertices of $H(\mathcal{S})$, we obtain that the number of edges is

$$\sum_{j=0}^{d-1} O(d^3 \cdot \min\{d, |B_{j+1}|\}) + \sum_{j=0}^{d-1} \sum_{r, z} O(d^2 \cdot \min\{d - |\text{Ch}(z, j)|, |\text{Ch}(r, z)|\}) + O(d) = O(d^3 \cdot \min\{n, d^2\}).$$

By the previous observations and Lemma 14 we obtain the following theorem.

Theorem 15. *An optimal leftmost canonical plan for an instance $\mathcal{S}$ of the TMP1R can be obtained by solving a shortest path problem in a weighted DAG with $O(d^3 \cdot \min\{n, d^2\})$ edge and $O(d^4)$ vertices.*

Corollary 16. *There exists an algorithm that, for each instance of the TMP1R that admits a finite cost solution, computes the cost of an optimal solution in time $O(n + d^3 \cdot \min\{n, d^2\})$.*

5 An approximation algorithm

In this section we consider a particular class of the leftmost canonical plans, that we call *sidestepping* plans, and give an $O(n^3)$ algorithm to compute the best sidestepping plan for an instance $\mathcal{S}$ of GMP1R on a tree. Then we prove that for each leftmost canonical plan A that solves an instance $\mathcal{S}$ of GMP1R on a tree there exists a sidestepping plan A^* that solves $\mathcal{S}$ and has at most $6d$ moves more than A . Thus, the cost of the minimal sidestepping plan for $\mathcal{S}$ is within a factor 7 of the cost of the optimal plan for $\mathcal{S}$.

Let us formally define the class of the sidestepping plans. We say that a leftmost canonical plan A is a *sidestepping* plan if the following additional property holds for all pairs of *adjacent* branch vertices v_j and v_z (i.e., vertices such that the path from v_j to v_z contains no other branch vertex): obstacles are moved backward to $\text{Ch}(z, j)$ only while the robot is at the sidestep vertex of v_j .

As done before, we can define the notion of left and right part of a plan and the notion of a class of left parts. However, by the restriction imposed on sidestepping plans we are only interested in classes $\mathcal{C}(j, k, l, r, b)$ where r is the index of the rightmost branch vertex behind v_j . In other words, we need to consider only $O(d^3)$ classes. Following the same approach given in Section 4 we reduce the problem of computing the minimal sidestepping plan for $\mathcal{S}$ to a shortest path problem on a weighted DAG that has $O(d^3)$ vertices and $O(d^2 \cdot \min\{n, d^2\})$ edges.

The next lemma shows how a leftmost canonical plan A is transformed into a sidestepping plan A^* that has at most $6d$ moves more than A . Due to lack of space the proof is omitted.

Lemma 17. *Let $\mathcal{S}$ be an instance of the TMP1R problem and A be a leftmost canonical plan for $\mathcal{S}$. Then there exists a sidestepping plan A^* that solves $\mathcal{S}$ and has at most $6d$ moves more than A .*

Theorem 18. *For each instance $\mathcal{S}$ of the GMP1R on a tree that admits a finite cost solution there exists a sidestepping plan that solves $\mathcal{S}$ whose cost is not greater than 7 times the cost of the optimal canonical plan for $\mathcal{S}$.*

Proof. By Lemma 17 there exists a sidestepping plan for $\mathcal{S}$ that has at most $6d$ moves more than the optimal plan for $\mathcal{S}$. Since the optimal plan for $\mathcal{S}$ has cost greater than or equal to d , we obtain that the cost of the optimal sidestepping plan for $\mathcal{S}$ is within a factor 7 of the cost of the optimal plan for $\mathcal{S}$.

6 Extensions and Open Problems

Our approach to robot motion planning allows us to solve also a series of related problems with essentially the same algorithm. We discuss a few extensions in the sequel.

Motion planning on a weighted tree. In this case we try to minimize the total distance travelled by the robot and the obstacles. As already observed in [3], in this case the optimal plan might not be canonical anymore and makes *wiggles*. Our approach can be used to obtain a $O(n^4)$ for the GMP1R on a weighted tree.

Vertices with capacities. In [3], it is considered the case in which vertices have integral capacities representing the number of objects (robots, and obstacles) that may sit on it at one time. They show that this problem can be reduced to the TMP1R (with no capacities) on a tree with $O(mn)$ vertices (m is the maximum capacity of a vertex). Thus, using the reduction directly, we would get a $O(n^4m^4)$ algorithm for the TMP1R with capacities. A more careful approach can take the running time of the algorithm to $O(n^4m^3)$.

Large robots. In some situations the robot or the obstacles are larger than the space in a vertex and thus sit at the same time on a number k of adjacent vertices (e.g., the robot is a train that has to move on tracks to reach a destination and other trains are sitting idle on the tracks). Our algorithm can handle this case with essentially the same running time.

Planning a trip: an open problem. The following generalization of the TMP1R problem seems worth investigating. We are given a list $t_1, t_2, \dots, t_l$ of vertices of the tree and a source vertex and have to plan a trip of the robot that starting from a specified source vertex s visits the vertices $t_1, t_2, \dots, t_l$ in this order. The problem is made interesting by the fact that when moving obstacles we have to take into account that the robot might need to go through that vertex in one of the next legs of the trip.

References

1. V. Auletta, D. Parente, G. Persiano *Optimal Planning of Robot Motion on a Tree with Obstacles*, Technical Report Università di Salerno, 1995 (a postscript file of this report can be downloaded at the URL <http://www.unisa.it/papers/robot.ps.gz>).
2. D. Kornhauser, G. Miller, and P. Spirakis, *Coordinating Pebble Motion on Graphs, the Diameter of Permutations Groups, and Applications*, in Proc. of 25-th IEEE Symp. on Found. of Comp. Sc., (FOCS), 241–250, 1984.
3. C. Papadimitriou, P. Raghavan, M. Sudan and H. Tamaki, *Motion Planning on a Graph*, in Proc. of 35-th IEEE Symp. on Found. of Comp. Sc., (FOCS), 511–520, 1994.
4. D. Ratner and M. Warmuth, *Finding a Shortest Solution for the $(N \times N)$ -Extension of the 15 Puzzle is NP-hard*, Journal of Symbolic Computation, 10:111–137, 1990.
5. J.T. Schwartz, M. Sharir, and J. Hopcroft, *Planning, Geometry, and Complexity of Robot Motion*, Ablex, Norwood NJ, 1987.